Gonzalez Vazquez Rodrigo

"Gestores de bases de datos distribuidas"

El diseño de una base de datos distribuida (BDD) requiere tomar decisiones sobre la colocación de datos y programas en los sitios de una red de computadoras. El enfoque top-down, más adecuado para sistemas homogéneos e integrados, sigue las siguientes etapas:

Etapa	Descripción	Resultado
Análisis de Requisitos	Define el entorno, los patrones de acceso y los objetivos de rendimiento, confiabilidad y escalabilidad.	Documento de requisitos
Diseño Conceptual	Determina entidades, atributos y relaciones del modelo de la empresa, independiente de la distribución.	Esquema Conceptual Global (GCS)
Diseño de Distribución	Fragmenta las relaciones y asigna fragmentos a los sitios de la red. Es el núcleo del diseño distribuido.	Esquemas Conceptuales Locales (LCS)
Diseño Físico	Mapea los LCS a las estructuras físicas de almacenamiento en cada sitio.	Esquemas Internos Locales (LIS)
Monitoreo y Ajuste	Observa el comportamiento del sistema y retroalimenta para rediseño si cambian los patrones de acceso.	Retroalimentación continua

En lugar de distribuir relaciones completas, los SGBDD dividen las relaciones en sub-relaciones llamadas fragmentos, que son la unidad básica de distribución. Las razones son:

- Las vistas de las aplicaciones son típicamente subconjuntos de relaciones, no relaciones completas; la localidad de acceso se define sobre esos subconjuntos.
- Distribuir relaciones enteras sin fragmentar obliga a escoger entre replicación innecesaria (con costosas actualizaciones) o elevado volumen de accesos remotos.
- La fragmentación permite la ejecución concurrente de múltiples transacciones (intraquery concurrency) al dividir una consulta en subconsultas que operan sobre fragmentos en paralelo.
- La fragmentación adecuada puede mejorar la seguridad y privacidad de los datos al mantener atributos sensibles en sitios específicos.

La fragmentación puede generar costos elevados si las aplicaciones requieren datos de múltiples fragmentos (joins distribuidos). Además, la verificación de restricciones de integridad que involucran atributos de distintos fragmentos incrementa la complejidad.

Fragmentación Horizontal

La fragmentación horizontal divide una relación en subconjuntos de tuplas (filas). Cada fragmento contiene todas las columnas de la relación original (o las necesarias para la reconstrucción) pero sólo un subconjunto de las filas. La reconstrucción se realiza mediante la operación UNION.

Fragmentación Vertical

La fragmentación vertical divide una relación en subconjuntos de atributos (columnas). Cada fragmento debe incluir la clave primaria de la relación para garantizar la reconstrucción mediante la operación JOIN. Se busca agrupar en el mismo fragmento los atributos que son accedidos conjuntamente por las mismas aplicaciones.

Fragmentación Híbrida (Mixta)

La fragmentación híbrida combina la fragmentación horizontal y vertical. Es la técnica más flexible y genera fragmentos con la información mínima necesaria para las aplicaciones. Se puede aplicar de dos formas:

Horizontal → Vertical: Primero se fragmenta horizontalmente, luego algunos de esos fragmentos horizontales se fragmentan verticalmente.

Vertical → Horizontal: Primero se fragmenta verticalmente, luego se fragmentan horizontalmente algunos fragmentos verticales.

La reconstrucción de la relación original es más costosa en fragmentaciones híbridas, pues requiere una combinación de operaciones UNION y JOIN. Sin embargo, es el esquema más utilizado en sistemas reales ya que permite mayor alineamiento con los patrones de acceso de las aplicaciones.

Comparativa de los Tipos de Fragmentación

Característica	Horizontal	Vertical	Híbrida
Unidad de partición	Tuplas (filas)	Atributos (columnas)	Tuplas y atributos
Operador de reconstrucción	UNION	JOIN (natural)	UNION + JOIN
Clave primaria en fragmentos	En todos (como cualquier atributo)	Repetida en todos (obligatorio)	Depende del orden de aplicación
Adecuada para	Datos con distribución geográfica o lógica por categorías	Atributos con distintas frecuencias de acceso o privacidad	Escenarios complejos con ambos tipos de partición
Riesgo principal	Joins distribuidos costosos si las apps necesitan datos de varios fragmentos	Joins costosos si atributos relacionados quedan en distintos fragmentos	Alta complejidad de gestión y reconstrucción
Ejemplo de uso	Empleados por sede, pedidos por región	Datos financieros separados de datos descriptivos	Empleados por sede y sólo atributos relevantes por sede

Replicación

La replicación de datos es el proceso de almacenar copias separadas de la base de datos (o de fragmentos de ella) en dos o más sitios de la red. Es la principal técnica de tolerancia a fallos en los SGBDD. Los objetivos principales son:

- Fiabilidad: si un sitio falla, el sistema continúa operando usando la copia disponible en otro sitio.
- Disponibilidad: las réplicas aseguran acceso continuo a los datos, incluso ante fallos parciales de red.
- Rendimiento en lectura: las consultas de sólo lectura pueden ejecutarse localmente o en la réplica más cercana, reduciendo la latencia y el tráfico de red.
- Reducción de carga de red: los datos se actualizan en réplicas durante horas de menor actividad; durante horas pico se procesan localmente.

Replicación Eager (Síncrona)

En la replicación eager (también llamada síncrona, activa o pesimista), las actualizaciones se propagan y se aplican a todas las réplicas dentro de los límites de la misma transacción. El cliente espera a que todas las réplicas confirmen la actualización antes de que la transacción se considere completada.

Aspecto	Descripción
Garantía de consistencia	Fuerte — todas las réplicas siempre contienen el mismo valor (consistencia mutua instantánea).
Mecanismo típico	Two-Phase Commit (2PC) combinado con broadcast atómico (VSCAST) para coordinación entre réplicas.
Latencia	Alta — el cliente espera la confirmación de todos los nodos; la latencia aumenta con la distancia.
Tolerancia a fallos	Media — si algún nodo falla durante la transacción, se necesita resolver el bloqueo.
Conflictos	Se previenen antes del commit: el sistema detecta conflictos y aborta una de las transacciones.

Aspecto	Descripción
Uso típico	Sistemas financieros, bancarios, reservas de vuelos — donde la consistencia es más importante que el rendimiento.

Replicación Lazy (Asíncrona)

En la replicación lazy (también llamada asíncrona, pasiva o perezosa), la transacción actualiza primero la copia maestra (master) y retorna al cliente. Las actualizaciones se propagan asíncronamente a las demás réplicas después de que la transacción concluye.

Aspecto	Descripción
Garantía de consistencia	Débil / eventual — las réplicas pueden divergir temporalmente; se converge tras la propagación.
Mecanismo típico	Log de transacciones + propagación diferida; multiversion concurrency control (MVCC).
Latencia	Baja — el cliente recibe confirmación inmediatamente tras actualizar la copia local/master.
Tolerancia a fallos	Alta — el sistema puede operar aunque réplicas estén desconectadas; se reconcilia al reconectar.
Conflictos	Se resuelven post-commit: last-write-wins, lógica de aplicación o fusión de actualizaciones concurrentes.
Uso típico	Sistemas web distribuidos, CDNs, aplicaciones móviles, DNS — donde la disponibilidad prima sobre la consistencia.